

Question 1

In natural language processing, stop words are frequently used words such as "a", "is", and "the" that are usually filtered out during text processing.

Write a function that accepts a string *text* and an integer *k*, and returns a list of words that appear at least *k* times in the text. The words should be returned in the order of their first occurrence in the text.

Example

Suppose *text* = "a mouse is smaller than a dog but a dog is stronger" and *k*=2

Output: ["a", "is", "dog"]

Explanation:

In order of appearance, the words that occur at least *k* = 2 times are "a", "is", and "dog".

Constraints

- *text* has at most 50000 characters.
 - Every character in *text* is either an English lowercase letter or a space.
 - *text* starts and ends with a letter. No two consecutive characters are spaces, i.e., *text* is a valid sentence.
 - There is at least one stop word in *text*.
 - $1 \leq k \leq 18$
-

Question 2

A janitor must carry waste bags to an outside trash bin. Each bag:

- Weighs between 1.01 and 3.00 pounds
- Can be carried along with other bags as long as the total weight ≤ 3.00 pounds

You are given an array *weight[]* where *weight[i]* is the weight of the *i*th bag.

Your task is to:

- Determine the minimum number of trips needed to carry all bags
- Each trip may include one or more bags, but their combined weight must not exceed 3.00

Return the minimum number of trips required.

Example

Suppose there are 5 bags with *weight* = [1.01, 1.99, 2.5, 1.5, 1.01]

Output: 3

Explanation:

1. First trip: bags with weights 1.01 and 1.99 (total: 3.00 pounds)
2. Second trip: bag with weight 2.5 (total: 2.50 pounds)
3. Third trip: bags with weights 1.5 and 1.01 (total: 2.51 pounds)

Constraints

- $1 \leq \text{size of } \textit{weight[]} \leq 1000$
- $1.01 \leq \textit{weight}[i] \leq 3.0$

Test Case Input Format

The first line contains the integer *n*, the size of *weight[]*.

The next *n* lines contain a float element of *weight[]*.

Question 3

Implement the transition logic for an ATM system that follows a state machine design pattern. The ATM has two states: unauthorized (initial state) and authorized (after successful login).

Transition Requirements

Define the *transition_table* variable as a Dictionary where:

- Keys are *State* objects
- Values are lists of transitions supported in that state.

Each transition is a 3-element tuple: (*action_name*, *checker*, *next_state*)

The checker function has this signature:

```
checker(action_param: Optional, atm_password: str, atm_current_balance: int) -> Tuple[bool, int, Optional]
```

The returned tuple contains:

1. A boolean indicating if the transition should be performed
2. The updated ATM balance after the transition
3. An optional return value (only used for balance inquiries)

Supported Actions

In the unauthorized state:

- *login* <password> - Transitions to authorized state if password is correct

In the authorized state:

- *logout* - Transitions back to unauthorized state
- *deposit* <amount> - Adds the specified amount to the balance
- *withdraw* <amount> - Deducts the specified amount if sufficient funds exist
- *balance* - Returns the current balance

Implementation Context

The ATM class is already provided:

```
class ATM:
    def __init__(self, init_state: State, init_balance: int, password: str, transition_table: Dict):
        self.state = init_state
        self._balance = init_balance
        self._password = password
        self._transition_table = transition_table

    def next(self, action: Action, param: Optional) -> Tuple[bool, Optional]:
        try:
            for transition_action, check, next_state in self._transition_table[self.state]:
                if action == transition_action:
                    passed, new_balance, res = check(param, self._password, self._balance)
                    if passed:
                        self._balance = new_balance
                        self.state = next_state
                        return True, res
            except KeyError:
                pass
            return False, None
```

Your implementation will be tested by a provided code stub on several input files. Each input file contains the password for the ATM and its initial balance. Then, it contains descriptions of the actions that will be performed on the ATM by the provided code. The result of each transition is printed to the standard output by the provided code.

▼ Input Format Format for Custom Testing

In the first line, there is a single string, *password*, the correct password.

In the second line, there is a single integer, *init_balance*, the initial balance of the ATM.

In the third line, there is a single integer, *q*, the number of actions the ATM will be tested on.

Each of the next, *q* lines describes an action request.

▼ Sample Case 0

Sample Input

```
hacker
10
8
withdraw 5
login foo
login hacker
withdraw 15
deposit 20
withdraw 15
balance
logout
```

Sample Output

```
Success=False unauthorized
Success=False unauthorized
Success=True authorized
Success=False authorized
Success=True authorized
```

```
Success=True authorized
Success=True authorized 15
Success=True unauthorized
```

Explanation

The ATM's password is "hacker" and its initial balance is 10. There are 8 actions to perform. The first action tries to withdraw 5 from the ATM but since it always starts in the "unauthorized" state, this action is unsuccessful. The second action tries to log in with the password "foo". This does not match the ATM's password, so the action is also unsuccessful. The third action tries to log in with the correct password "hacker". This action is successful, so the ATM moves to the "authorized" state. The next action tries to withdraw 15 from the ATM but this amount is greater than the current balance of the ATM, so the action is unsuccessful. The fourth action deposits 20 into the ATM, so its current balance is now $10 + 20 = 30$. The next action, again, tries to withdraw 15, and it successfully changes the balance of the ATM to $30 - 15 = 15$. The next action gets the current balance of the ATM, i.e., 15. The last action logs out, so the ATM moves to the "unauthorized" state.

▼ Sample Case 1

Sample Input

```
somepass
100
5
logout
login somepass
balance
login somepass
login wrongpass
```

Sample Output

```
Success=False unauthorized
Success=True authorized
Success=True authorized 100
Success=False authorized
Success=False authorized
```

Explanation

The ATM's password is "somepass" and its initial balance is 100. There are 5 actions to be performed. The first action tries to log out, but since the ATM always starts in the "unauthorized" state, this action is unsuccessful. The second action tries to log in with the correct password "somepass". This action is successful, so the ATM moves to the "authorized" state. The next action gets the balance of the ATM, so 100 is returned. The next two actions both try to log in, but since the ATM is already in the "authorized" state, both of them are unsuccessful.